



UNIVERSITY OF TEHRAN

College of Interdisciplinary Science and Technologies

Soft Computing

Retrieval-Augmented Image Captioning

Name: Mahan Madani

Student ID: 830504035

Submission Date: February 14, 2026

Contents

1	Introduction	3
1.1	Reference Paper	3
2	Project Architecture	4
2.1	Modules	4
2.2	Notebooks	4
3	Dataset	5
4	Multimodal Retriever	5
4.1	Retrieval Module	5
4.2	FAISS Vector Databases	6
5	Custom Vision-Language Model (FusionVLM)	7
5.1	Model Architecture	7
5.2	Fusion Blocks	9
5.3	Using LoRA for Finetuning	9
5.4	Model Training	10
5.5	Evaluation Metrics	11
6	Inference Results and Comparison with BLIP	12
7	Conclusion	13
	References	14

List of Figures

1	The framework of Re-ViLM	3
2	Example of Similar Image Retrieval	5
3	Retrieving Similar Images using the FAISS Index	7
4	The Architecture of the Custom Vision-Language Model	7
5	Model Parameters After Applying LoRA	10
6	Train and Validation Loss Over Epochs	10
7	Evaluation Metrics Over Epochs (validation data)	11
8	Sample Images Used for Inference (Source: Train Dataset)	12
9	Sample Images Used for Inference (Source: Test Dataset)	12

List of Tables

1	Sample Captions for train data images	12
2	Sample Captions for test data images	12
3	Evaluation results of FusionVLM and BLIP	13

1. Introduction

The goal of this project is to build and train a Retrieval-Augmented Vision-Language Model (VLM) for Image Captioning, which enhances captioning performance by incorporating an external retrieval database. This project is inspired by the ideas introduced in the Retrieval-Augmented Visual Language Model for Zero and Few-Shot Image Captioning (**Re-ViLM**) proposed by Yang et al. [1].

For training and evaluation, the **Flickr30k** image captioning dataset [2] is used, a widely adopted benchmark in vision-language research.

The final trained model is compared against the **BLIP** framework (Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation) introduced by Li et al. [3], which has been specifically trained for image captioning.

1.1 Reference Paper

Re-ViLM (Retrieval-Augmented Visual Language Model) is a framework designed to improve zero-shot and few-shot image captioning by augmenting a vision-language model with external retrieval. Instead of relying solely on parametric knowledge stored in the model weights, Re-ViLM retrieves visually or semantically similar image-caption pairs from a large external database and conditions the caption generation process on this additional context.

The retrieved examples provide grounded linguistic patterns and object-level cues that guide the model toward more accurate and detailed descriptions, particularly in scenarios where labeled training data is limited. By combining retrieval with generation, Re-ViLM effectively reduces hallucinations and enhances generalization, demonstrating that non-parametric memory can significantly strengthen captioning performance without requiring full model retraining.

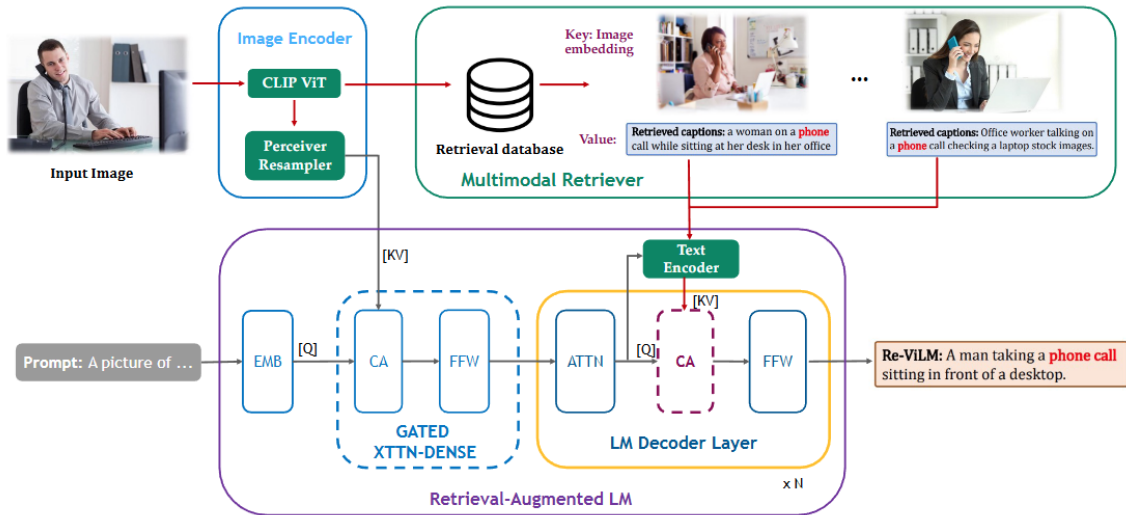


Figure 1: The framework of Re-ViLM

As shown in **Figure 1**, the Re-ViLM model first extracts CLIP embeddings of the input image, and uses it to retrieve similar image-text pairs from the database. Within some predetermined layers, the retrieval-augmented LM will cross-attend the visual representation from the image encoder, and the textual representation from the text encoder, which encodes the retrieved captions.

2. Project Architecture

The project comprises several modules that are used for loading the custom VLM model and the data, creating FAISS vector databases, training the model, and evaluating it. The contents of these modules are used in 5 Jupyter notebooks that should be run in order of their names, handling all the major tasks in the project.

2.1 Modules

Below you can find a list of all the files found in the `Modules` package, alongside a brief description of their contents.

- `config.py` : Contains various constants, file paths, and configurations used throughout the project.
- `datasets.py` : A collection of dataset classes for finding image embeddings and training the vision-language model.
- `embedding_module.py` : Contains a wrapper class for creating CLIP embeddings for images and text.
- `FusionVLM.py` : The Custom Vision-Language Model (titled FusionVLM) is implemented in this module. Several utility functions (such as for saving the model or loading weights) are also provided.
- `retrieval_module.py` : Contains the Retriever class used for Multimodal retrieval of images and text
- `train_VLM.py` : Contains the functions for training and evaluating the model.
- `metrics.py` : Implementation of the various metrics (BLEU, METEOR, etc.) used for evaluation is located in this module.
- `inference.py` : Contains functions for captioning images using FusionVLM and BLIP.
- `utils.py` : A collection of functions shared by other modules.

2.2 Notebooks

- `01-data_split.ipynb` : Split the data using a fixed seed for reproducibility
- `02-FAISS_image_index.ipynb` : Find CLIP embeddings of all **images** in the train dataset and store them in a FAISS vector database
- `03-FAISS_caption_index.ipynb` : Find CLIP embeddings of all **captions** in the train dataset and store them in a FAISS vector database. Note that our dataset has multiple captions per image.
- `04-FusionVLM.ipynb` : Train the custom VLM (FusionVLM) and evaluate its results.
- `05-Inference_and_evaluation.ipynb` : Perform image captioning on sample images from the test and train dataset and compare the results.

Note: The Jupyter notebook files must be run in order to split the dataset, build FAISS databases, and create metadata files.

3. Dataset

The dataset used for this project is the **Flickr30k** dataset [2], a widely used benchmark in vision-and-language research that contains **31,783** images, each paired with **5** human-written captions. Having several descriptions per image gives models diverse linguistic perspectives on the same visual content, which helps them learn robust associations between visual features and textual descriptions.

The images in the dataset depict a wide variety of everyday scenes and objects, and the captions are human-generated. Because of this, the Flickr30k dataset provides rich real-world samples that are suitable for training and evaluating image captioning models that must generalize well to novel inputs.

The dataset must be manually downloaded and placed in the path specified in the config file.

Vision Transformers (ViTs), which are used in our custom VLM, require large amounts of training data to generalize well. This is because unlike convolutional neural networks (CNNs), they do not encode strong inductive biases such as locality and translation equivariance. Instead, ViTs rely on self-attention mechanisms to learn global relationships between image patches.

This flexibility allows ViTs to model long-range dependencies effectively. It also means they must learn many visual patterns from scratch, making them more data-hungry in practice. To support this requirement, the Flickr30k dataset is split such that **1,000 images** are reserved for validation and testing, while the remaining images (over **31,000**) are used for training the model.

4. Multimodal Retriever

The retrieval module encodes all images and captions into a shared embedding space using **CLIP** (Contrastive Language–Image Pretraining) [5], allowing direct comparison between visual and textual representations. Given an input image embedding, it computes similarity scores (using cosine similarity) against the precomputed embeddings and returns the most similar images and their associated captions.

4.1 Retrieval Module



Figure 2: Example of Similar Image Retrieval

The retriever module builds 2 separate vector databases: the **Image Embedding Database** and the **Caption Embedding Database**. The first is used for storing CLIP embeddings of all images in the *training dataset*, and the second for storing CLIP embeddings of all captions in the *training dataset*. **The module does not store any embeddings from the test dataset.**

Each vector database is accompanied by a **metadata JSON** file, which includes details such as the name of the image and its corresponding captions for the image embedding database. For the captioning embedding database, the plaintext version of the caption and the name of its corresponding image are stored.

Note that similar images and captions are retrieved from the databases at the time of inference. During training however, a list of the top similar images and captions are also saved in the metadata file for each image. This is done in order to increase performance and to avoid rerunning the database search code repeatedly in every epoch. The retriever also ensures any retrieved captions do not belong to the query image during training.

Figure 2 demonstrates one example of similar image Retrieval. Additionally, the following **similar captions** were retrieved from the captions database:

- A small boy toys on the floor and a Christmas tree behind him holds a bright yellow plastic shovel pointed at the camera.
- A child next to a Christmas tree playing with brooms.
- A little boy is playing with a yellow shovel in front of a Christmas tree.

Compared to the actual **ground truth captions** of the query image:

- A picture of Young boy holding a light saber in front of a Christmas tree with wrapping paper littering the floor.
- A picture of Boy indoors by Christmas tree playing with green light-up sword toy.
- A picture of A boy holds a light saber in front of a Christmas tree.
- A picture of A boy receives a light saber for Christmas.
- A picture of A boy looks at a new toy during Christmas.

4.2 FAISS Vector Databases

FAISS (Facebook AI Similarity Search) [4] is a library for efficient similarity search and clustering of dense vector representations. It is designed to scale to very large datasets, supporting both exact and approximate nearest neighbor search using optimized indexing structures such as IVF (Inverted File Index) and product quantization.

FAISS provides GPU acceleration and highly optimized implementations for high-dimensional vector retrieval, making it particularly suitable for retrieval-augmented vision–language systems. In this project, FAISS is used as the vector database for the **image embeddings database** and the **caption embedding dataset**.

Figure 3 displays examples of using the FAISS image index for finding similar images from the training dataset, using an image from the test dataset as the **query**. The CLIP embedding of the query is first computed, then compared to all other embeddings in the database using cosine similarity. The FAISS index returns a list of the indices of the most similar images, alongside their distances.



Figure 3: Retrieving Similar Images using the FAISS Index

5. Custom Vision-Language Model (FusionVLM)

The custom Vision-Language model proposed in this project (**FusionVLM**) utilizes pretrained vision and text encoders, performs cross-attention between images and text using custom Fusion Blocks, and finally generates a caption for the query image via a fine-tuned text-decoder.

In this section, we will go through the **architecture** of the model, the model **training process**, and the **metrics** used to evaluate the model.

5.1 Model Architecture

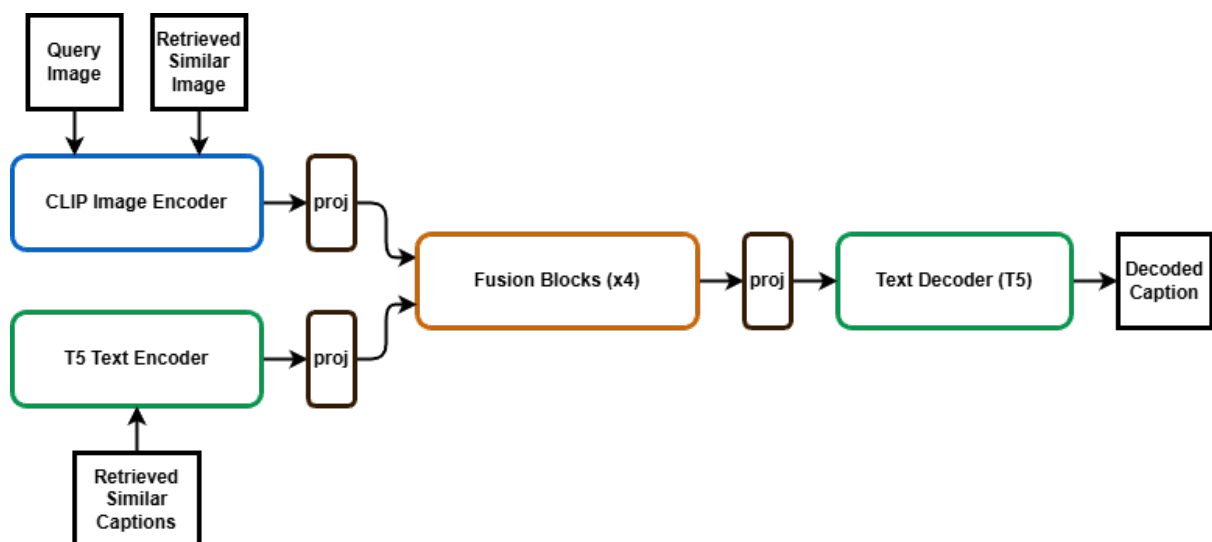


Figure 4: The Architecture of the Custom Vision-Language Model

Diagram created using the free online tool *app.diagrams.net*

The model comprises 6 main components, as shown in **Figure 4**. These components are:

- **CLIP Image Encoder:** Used as the embedding layer for the query image and the n most similar images retrieved from the database. The **last hidden state** of the model is used as the output, which means each image is embedded as a tensor of shape $[T_v, D_v]$, where T_v is the sequence length (number of input patches) and D_v is the hidden dimension of the vision encoder.

The query image embedding and the retrieved image embeddings are stacked on each other, resulting in a final tensor of size $[(n + 1) * T_v, D_v]$

- **T5 Text Encoder:** Used as the embedding layer for the k most similar captions to the query image, retrieved from the database. The captions are combined together to create **one context string**, which is then embedded. Similar to the image embeddings, the **last hidden state** of the encoder is used as output.

This results in a tensor of size $[T_t, D_t]$, where T_t is the sequence length and D_t is the hidden dimension of the text encoder.

- **Input Projection Layers:** The vision and text embeddings are transformed to the appropriate hidden size of the fusion blocks, changing their dimension D to be the same size as D_f , which in this project is **768**.

This results in the text hidden representation shape to be $[T_t, D_f]$ and vision hidden representation shape to be $[(n + 1) * T_v, D_f]$

- **Fusion Blocks:** The fusion blocks perform **cross-attention** between image and text hidden representations and outputs both resulting tensors as attended embeddings to the next block. Additional information regarding the fusion blocks can be found in a later section.
- **Text Encoder Projection:** After being processed by the fusion blocks, **the vision hidden representation is discarded** and the **text hidden representation is projected** to the same space as the text decoder to prepare for caption generation.

The output tensor will be of shape $[T_t, D_{decoder}]$.

- **T5 Text Decoder:** The text decoder is a full T5 encoder-decoder model without the initial embedding layer, as we pass already embedded text tensors to it. The decoder autoregressively generates the caption for the image. The generated caption is decoded using the T5 Tokenizer into natural language text.

Note that the implemented version of FusionVLM uses the following pretrained models:

Vision Encoder: CLIP "openai/clip-vit-base-patch32" CLIP (Contrastive Language–Image Pre-training) [5] is a vision–language model trained using a contrastive objective on large-scale image–text pairs collected from the web. Instead of training on a fixed set of labeled classes, CLIP learns to align visual and textual representations in a shared embedding space, enabling strong zero-shot transfer to downstream tasks.

The model consists of an image encoder (the Vision Transformer in the **ViT-B/32** used in this project) and a text encoder trained jointly to maximize similarity between matched image–text pairs while minimizing similarity to mismatched pairs. This training strategy allows CLIP to generalize to novel concepts without task-specific fine-tuning, making it a powerful backbone for vision–language applications.

Text Encoder and Decoder: T5 "t5-base" T5 (Text-to-Text Transfer Transformer) [6] is a unified sequence-to-sequence Transformer model that formulates all NLP tasks in a text-to-text framework. Both inputs and outputs are represented as text strings, enabling a single architecture to handle tasks such as translation, summarization, classification, and generation. Pre-trained on the large-scale C4 corpus using a span-corruption objective, T5 learns strong language modeling and conditional generation capabilities.

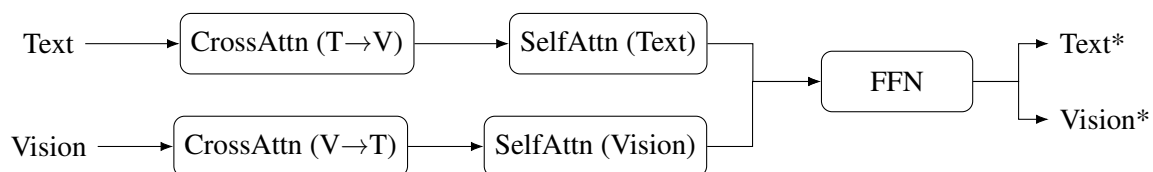
The **t5-base** variant used in this project provides a balanced trade-off between model capacity and computational efficiency, making it well-suited as a text decoder in generation systems.

5.2 Fusion Blocks

Fusion blocks are instances of the `BidirectionalFusionBlock` class, which is a custom transformer-based multimodal module designed to integrate (**fuse**) textual and visual representations before decoding.

Its core goal is to allow text and image embeddings to refine each other through structured attention, rather than simply concatenating or projecting them into a shared space. This is achieved through **symmetric cross-attention**: text attends to vision features, and vision attends to text features, enabling bidirectional information flow between modalities at every layer.

Architecturally, the block mirrors a standard Transformer encoder layer but duplicates the pathway for both modalities. It first performs **cross-attention in both directions**, followed by modality-specific **self-attention**, which refines each representation internally after cross-modal interaction. Finally, a shared **position-wise feed-forward network** transforms the features. Each stage is wrapped with **residual connections**, **layer normalization**, and **dropout**, stabilizing training and preserving gradient flow. The attention mask ensures that padded text tokens do not influence cross or self-attention computations.



A key strength of this design is that fusion is iterative: cross-modal alignment happens before self-refinement, meaning each modality continuously adapts to the other. Because the final **text hidden** representation has already absorbed visual context through multiple fusion blocks, it can be directly passed to a T5 decoder as a vision-aware textual embedding.

5.3 Using LoRA for Finetuning

Low-Rank Adaptation (LoRA) is a parameter-efficient finetuning method [7] that enables adapting large pretrained models to new tasks without updating all of their weights. Instead of modifying the full weight matrices, LoRA injects low-rank learnable matrices into existing layers, allowing task-specific adaptation with a small fraction of the total parameters.

This approach drastically reduces memory and storage requirements while maintaining comparable performance to full finetuning, making it suitable for large-scale transformers. A simple LoRA config is applied to the **T5 text decoder**'s self-attention and cross-attention layers, freezing all other weights except the language modeling head (`lm_head`).

Note that the **text and vision encoders** are frozen to leverage their pretrained knowledge while avoiding the cost of updating millions of additional parameters. Freezing stabilizes the embeddings and preserves general-purpose representations learned from large datasets.

The fusion blocks are fully trained because they are not initialized randomly without any pre-training. These blocks are responsible for cross-modal alignment, which is task-specific and requires full adaptability.

Figure 5 displays the number and state of the model parameters after applying LoRA to the decoder and freezing the encoders. The model is initialized with 4 Fusion blocks, each with 8 attention heads and a hidden dimension of 768.

Module	Total	Trainable	Frozen
vision_encoder	87,456,000	0	87,456,000
text_encoder	109,628,544	0	109,628,544
vision_proj	590,592	590,592	0
text_proj	590,592	590,592	0
fusion_blocks	56,724,480	56,724,480	0
post_fusion_ln	1,536	1,536	0
fusion_proj	590,592	590,592	0
text_decoder	254,655,744	31,752,192	222,903,552
TOTAL	510,238,080	90,249,984	419,988,096

Figure 5: Model Parameters After Applying LoRA

5.4 Model Training

Custom dataset and data collator classes are implemented for training the model. These classes store and provide the query image to be captioned, one retrieved image (the most similar from the training dataset), a context string comprised of the top 3 most similar captions, and the target label for the query image, which is chosen randomly from the 5 available captions and may change with every epoch.

The T5 model (which serves as the model’s text decoder and final layer) uses a **cross-entropy** loss over the target caption, which encourages the decoder to assign high probability to the correct next token at each step.

The model is optimized using Adam, with a learning rate of 10^{-4} and a weight decay of 10^{-2} . The model is trained for 20 epochs and evaluated on the validation data after each epoch.

Figure 6 displays the model’s loss values for the train and validation sets.

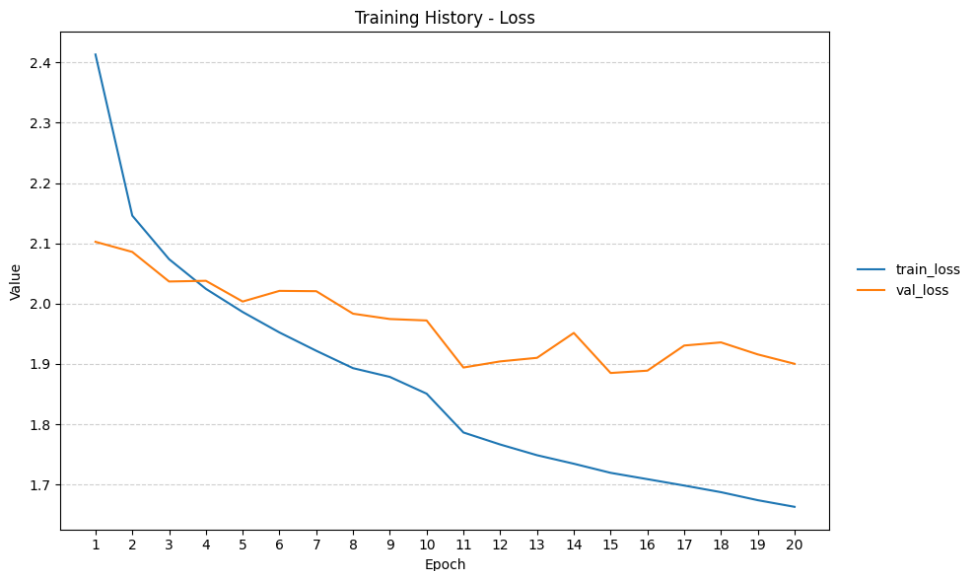


Figure 6: Train and Validation Loss Over Epochs

5.5 Evaluation Metrics

The following metrics are used to evaluate the model after each epoch:

- **BLEU-1, BLEU-2, BLEU-3, BLEU-4:** Measures n-gram precision between the generated captions and reference captions. BLEU-1 considers unigrams, BLEU-2 bigrams, BLEU-3 trigrams, and BLEU-4 four-grams, with higher scores indicating closer matches to human references.
- **METEOR:** Evaluates generated text by considering exact word matches, stemmed forms, synonyms, and word order. It is designed to correlate better with human judgments than BLEU, especially for sentence-level evaluation.
- **ROUGE-L:** Measures the longest common subsequence (LCS) between the generated caption and reference captions. It captures both precision and recall aspects, emphasizing fluency and overlap of sequence structure.
- **CIDEr:** Consensus-based Image Description Evaluation metric that computes cosine similarity of TF-IDF weighted n-grams between generated captions and multiple references. It emphasizes consensus among human references and is widely used for image captioning tasks.

Figure 7 displays the metric values for the validation dataset. For all metrics, a higher score indicates better text generation and captioning results.

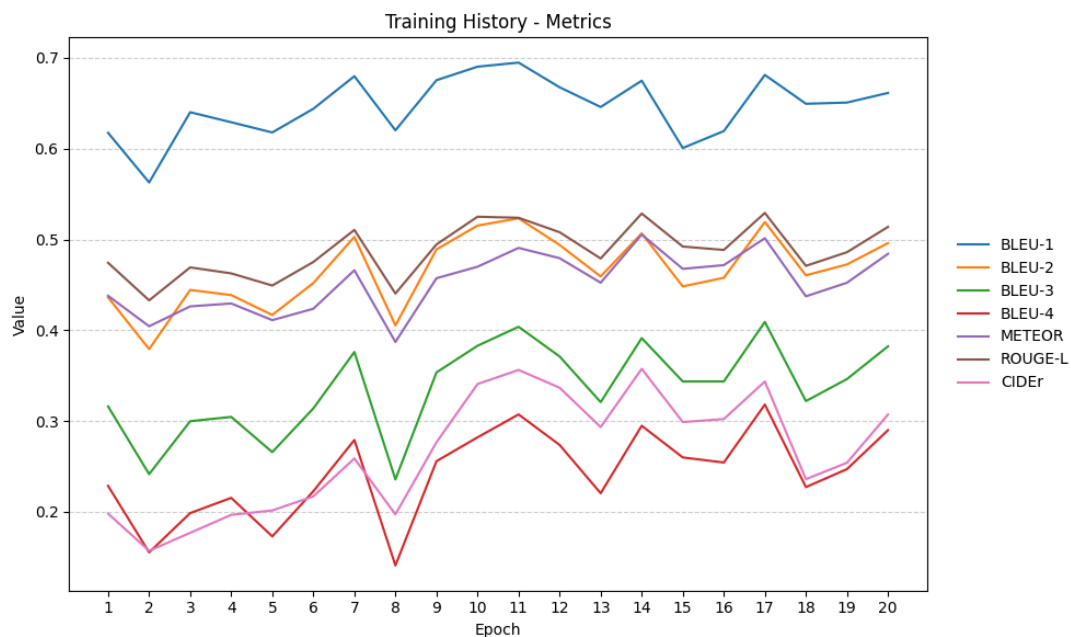


Figure 7: Evaluation Metrics Over Epochs (validation data)

The exact values for the loss and metrics of each epoch can be found in the `history0-20.json` file. Based on both of these plots, we can see the model reaches a point where it cannot generalize further, resulting in fluctuating loss and metric values for the validation set.

6. Inference Results and Comparison with BLIP

In order to evaluate our model more accurately, we compare the captioning results of our model with those of **BLIP model** "Salesforce/blip-image-captioning-base" [3].

Figure 8 and **Table 1** showcase sample images of the train dataset captioned.



Figure 8: Sample Images Used for Inference (Source: Train Dataset)

	Train Image 1	Train Image 2	Train Image 3
FusionVLM	A picture of a little boy on a sunny day	A picture of a man in a white shirt is walking in front of a building.	A picture of hikers in the mountains
BLIP	a small child standing in a field of grass	a crowd of people walking on a busy street at night	a man with a backpack

Table 1: Sample Captions for train data images

Figure 9 and **Table 2** showcase sample images of the test dataset captioned.



Figure 9: Sample Images Used for Inference (Source: Test Dataset)

	Test Image 1	Test Image 2	Test Image 3
FusionVLM	A picture of on a sunny day with a boat in the background	A picture of a man and a woman are walking down the street	A picture of soccer players in a stadium
BLIP	a large body of water	a group of people sitting at a table eating	a group of people on a soccer field

Table 2: Sample Captions for test data images

Table 3 displays the results of evaluating the two models on the validation set. Our model outperforms BLIP on the validation metrics, likely due to the fact that it has been trained specifically on images from the Flickr30k dataset, giving it prior exposure to the types of scenes and objects present in this dataset. In contrast, BLIP, while strong in general vision-language tasks, has not seen these images during training and therefore is less familiar with the distribution of the validation set, resulting in lower performance.

	BLEU-1	BLEU-2	BLEU-3	BLEU-4	METEOR	ROUGE-L	CIDEr
FusionVLM	0.6614	0.4960	0.3823	0.2899	0.4842	0.5138	0.3072
BLIP	0.3253	0.2462	0.1759	0.1183	0.2672	0.4204	0.2539

Table 3: Evaluation results of FusionVLM and BLIP

7. Conclusion

This project presented FusionVLM, a vision–language model that integrates frozen text and vision encoders with fully trainable fusion blocks and a LoRA-fine-tuned T5 decoder to enable efficient adaptation. The model achieves strong performance with significantly reduced computational cost compared to training a full VLM from scratch. Experimental results demonstrate improved validation metrics compared to BLIP, highlighting the effectiveness of the fusion strategy. Overall, this model shows that parameter-efficient fine-tuning combined with structured cross-modal learning provides a practical and scalable approach for vision–language generation tasks.

References

- [1] Zhuolin Yang, Wei Ping, Zihan Liu, Vijay Korthikanti, Weili Nie, De-An Huang, Linxi Fan, Zhiding Yu, Shiyi Lan, Bo Li, Mohammad Shoeybi, Ming-Yu Liu, Yuke Zhu, Bryan Catanzaro, Chaowei Xiao, and Anima Anandkumar, *Re-ViLM: Retrieval-Augmented Visual Language Model for Zero and Few-Shot Image Captioning*, In Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 11844–11857, 2023.
- [2] Peter Young, Alice Lai, Micah Hodosh, and Julia Hockenmaier, *From Image Descriptions to Visual Denotations: New similarity metrics for semantic inference over event descriptions*, Transactions of the Association for Computational Linguistics, vol. 2, pp. 67–78, 2014.
- [3] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi, *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*, Proceedings of the 39th International Conference on Machine Learning (PMLR 162), pp. 12888–12900, 2022.
- [4] Jeff Johnson, Matthijs Douze, and Hervé Jégou, *Billion-scale similarity search with GPUs*, IEEE Transactions on Big Data, 7(3):535–547, 2021.
- [5] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever, *Learning Transferable Visual Models From Natural Language Supervision*, Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR 139:8748–8763, 2021.
- [6] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu, *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*, Journal of Machine Learning Research, 21(140):1–67, 2020.
- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, and Weizhu Chen, *LoRA: Low-Rank Adaptation of Large Language Models*, arXiv preprint arXiv:2106.09685, 2021.